

1 Control de versions amb GIT

1.1 Sobre el control de versions

Què és el control de versions, i per què t'hauria d'importar? El control de versions és un sistema que registra els canvis realitzats sobre un arxiu o conjunt d'arxius al llarg del temps, de manera que puguis recuperar versions específiques més endavant. Per als exemples d'aquest llibre duràs control de versions de codi font, encara que en realitat pots fer això amb gairebé qualsevol tipus d'arxiu que trobis en un ordinador.

Si ets dissenyador gràfic o web, i vols mantenir cada versió d'una imatge o disseny (cosa que sens dubte vols), un sistema de control de versions (Version Control System o VCS en anglès) és una elecció molt sàvia. Et permet revertir arxius a un estat anterior, revertir el projecte sencer a un estat anterior, comparar canvis al llarg del temps, veure qui va modificar per última vegada alguna cosa que pot estar causant un problema, qui va introduir un error i quan, i molt més. Fer servir un VCS també significa generalment que si espatlles o perds arxius, els pots recuperar fàcilment. A més, obtens tots aquests beneficis a un cost molt baix.

Sistemes de control de versions locals

Un mètode de control de versions usat per molta gent és copiar els fitxers a un altre directori (potser indicant la data i hora en què ho van fer, si són espavilats). Aquest enfocament és molt comú perquè és molt simple, però també tremendament propens a errors. És fàcil oblidar en quin directori et trobes, i guardar accidentalment a l'arxiu equivocat o sobreescriure arxius que no volies.

Per fer front a aquest problema, els programadors van desenvolupar fa temps VCSs locals que contenen una simple base de dades en què es portava registre de tots els canvis realitzats sobre els arxius (vegeu Figura 1-1).

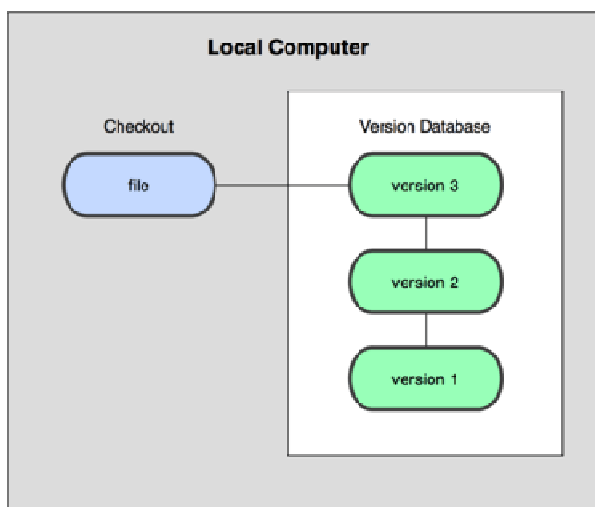


Figura 1-1. Diagrama de control de versions local.

Una de les eines de control de versions més popular va ser un sistema anomenat rcs, que encara podem trobar en molts dels ordinadors actuals. Fins i tot al famós sistema operatiu Mac OS X inclou l'ordre rcs quan instal·les les eines de desenvolupament. Aquesta eina funciona bàsicament guardant conjunts de pedaços (és a dir, les diferències entre arxius) d'una versió a una altra en un format especial en disc; pot llavors recrear com era un arxiu en qualsevol moment sumant els diferents pedaços.

Sistemes de control de versions centralitzats

El següent gran problema que es troba la gent és que necessiten col·laborar amb desenvolupadors en altres sistemes. Per solucionar aquest problema, es van desenvolupar els sistemes de control de versions centralitzats (Centralized Version Control Systems o CVCSs en anglès). Aquests sistemes, com CVS, Subversion, i Perforce, tenen un únic servidor que conté tots els arxius versionats, i diversos clients que descarreguen els arxius des d'aquest lloc central. Durant molts anys aquest ha estat l'estàndard per al control de versions (vegeu Figura 1-2).

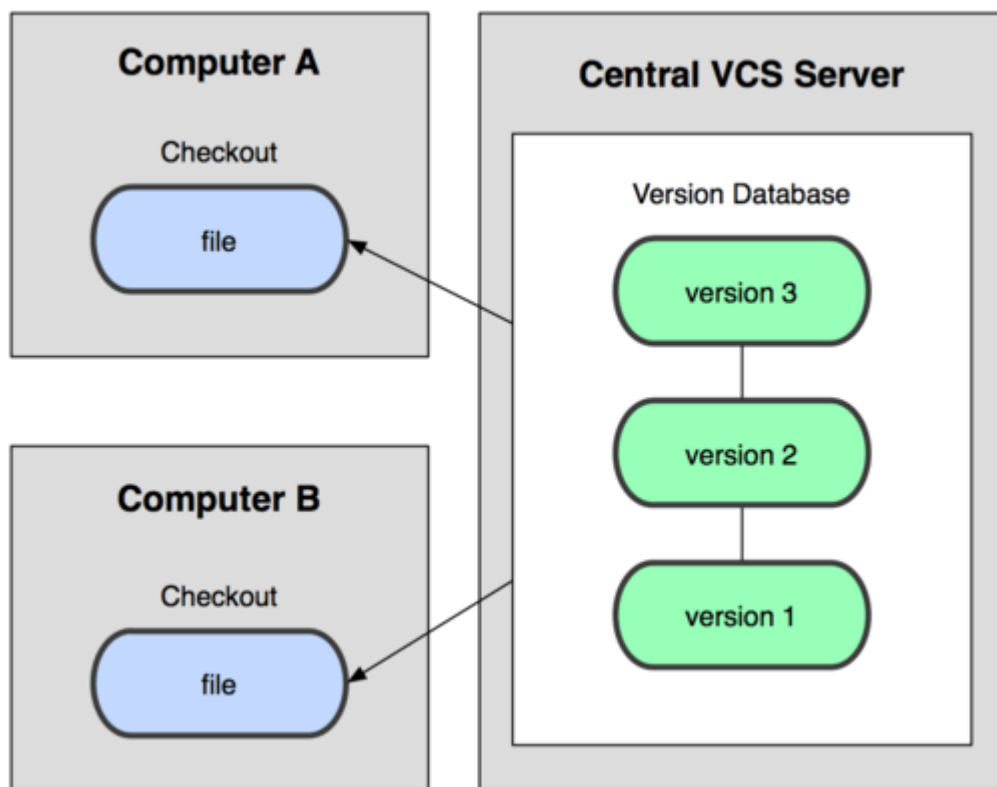


Figura 1-2. Diagrama de control de versions centralitzat.

Aquesta configuració ofereix molts avantatges, especialment davant VCSs locals. Per exemple, tothom pot saber (fins a cert punt) en què estan treballant els altres col·laboradors del projecte. Els administradors tenen control detallat de què pot fer cadascú, i és molt més fàcil administrar un CVCS de tenir de lluitar amb bases de dades locals en cada client.

No obstant això, aquesta configuració també té serioses desavantatges. La més òbvia és el punt únic de fallada que representa el servidor centralitzat. Si aquest servidor cau durant una hora, llavors durant aquesta hora ningú pot col·laborar o guardar canvis

versionats d'allò en què estan treballant. Si el disc dur en què es troba la base de dades central queda malmesa, i no s'han portat còpies de seguretat adequadament, perds absolutament tota la història del projecte excepte aquelles instantànies que la gent pugui tenir en les seves màquines locals. Els VCSs locals pateixen d'aquest mateix problema quan tens tota la història del projecte en un únic lloc, t'arrisques a perdre-ho tot.

Sistemes de control de versions distribuïts

És aquí on entren els sistemes de control de versions distribuïts (Distributed Version Control Systems o DVCSs en anglès). En un DVCS (com Git, Mercurial, Bazaar o darcs), els clients no només descarreguen l'última instantània dels arxius: repliquen completament el repositori. Així, si un servidor mor, i aquests sistemes estaven col·laborant a través d'ell, qualsevol dels repositoris dels clients pot copiar al servidor per restaurar-lo. Cada vegada que es descarrega una instantània, en realitat es fa una còpia de seguretat completa de totes les dades (vegeu Figura 1-3).

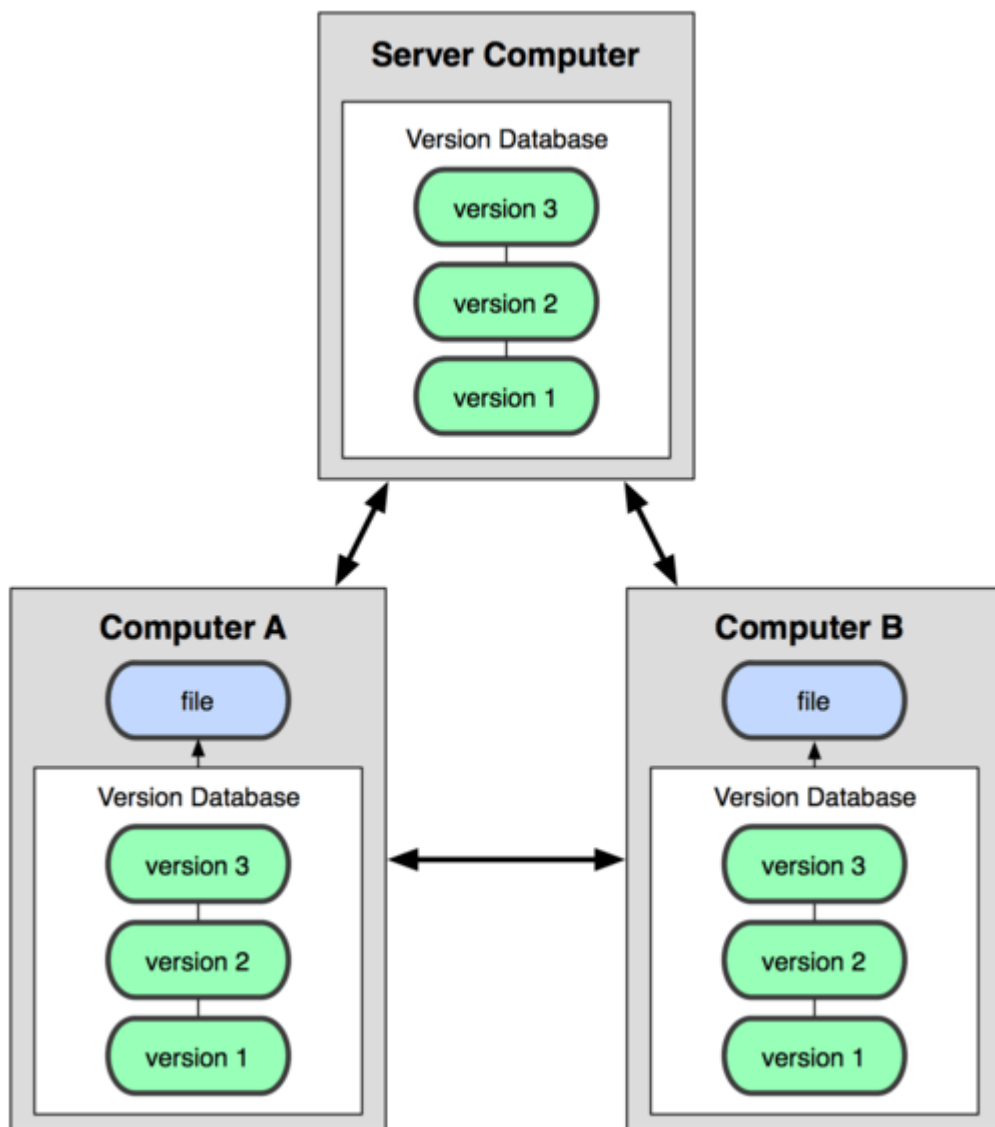


Figura 1-3. Diagrama de control de versions distribuït.

És més, molts d'aquests sistemes se les arreglen força bé tenint diversos dipòsits amb els quals treballar, de manera que pots col·laborar amb diferents grups de gent simultàniament dins del mateix projecte. Això et permet establir diversos fluxos de treball que no són possibles en sistemes centralitzats, com poden ser els models jeràrquics.

1.2 Una breu història de Git

El 2005, la relació entre la comunitat que desenvolupava el nucli de Linux i la companyia que desenvolupava BitKeeper es va enfonsar, i l'eina va deixar de ser oferta gratuïtament. Això va impulsar a la comunitat de desenvolupament de Linux (i en particular a Linus Torvalds, el creador de Linux) a desenvolupar la seva pròpia eina basada en algunes de les lliçons que van aprendre durant l'ús de BitKeeper. Alguns dels objectius del nou sistema van ser els següents:

- Velocitat
- Disseny senzill
- Fort suport al desenvolupament no lineal (milers de branques paral·leles)
- Completament distribuït
- Capaç de manejar grans projectes (com el nucli de Linux) de manera eficient (velocitat i mida de les dades)

Des del seu naixement el 2005, Git ha evolucionat i madurat per ser fàcil d'usar i encara conservar aquestes qualitats inicials. És tremendament ràpid, molt eficient amb grans projectes, i té un increïble sistema de ramificació (branching) per al desenvolupament no lineal.

1.3 Fonaments de Git

Llavors, què és Git en poques paraules? És molt important assimilar aquesta secció, perquè si entens el que és Git i els fonaments de com funciona, probablement et sigui molt més fàcil utilitzar Git de manera eficaç. A mesura que aprenguis Git, intenta oblidar tot el que puguis saber sobre altres VCSs, com Subversion i Perforce; fer-ho t'ajudarà a evitar confusions subtils a l'hora d'utilitzar l'eina. Git emmagatzema i modela la informació de forma molt diferent a aquests altres sistemes, tot i que la seva interfície sigui bastant similar; comprendre aquestes diferències evitarà que et confonguis a l'hora d'usar-lo.

Instantànies, no diferències

La principal diferència entre Git i qualsevol altre VCS (Subversion i companyia inclosos) és com Git modela les seves dades. Conceptualment, la majoria dels altres sistemes emmagatzemen la informació com una llista de canvis en els arxius. Aquests sistemes (CVS, Subversion, Perforce, Bazaar, etc.) Modelen la informació que emmagatzemen com un conjunt d'arxius i les modificacions fetes sobre cadascun d'ells al llarg del temps, com il·lustra la Figura 1-4.

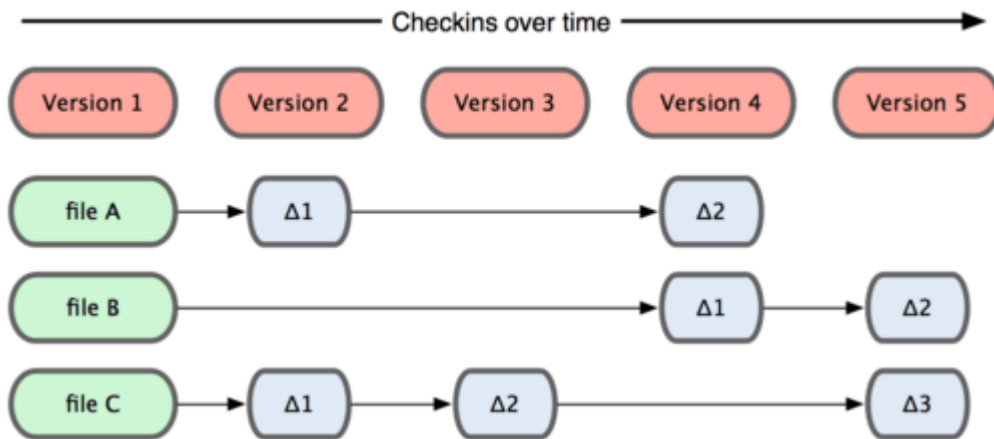


Figura 1-4. Altres sistemes tendeixen a emmagatzemar les dades com canvis de cada arxiu respecte a una versió base.

Git no modela ni emmagatzema les seves dades d'aquesta manera. En canvi, Git modela les seves dades més com un conjunt d'instantànies d'un mini sistema d'arxius. Cada vegada que confirmes un canvi, o guardes l'estat del teu projecte en Git, ell bàsicament fa una foto de l'aspecte de tots els teus arxius en aquest moment, i guarda una referència a aquesta instantània. Per ser eficient, si els arxius no s'han modificat, Git no emmagatzema l'arxiu de nou, només un enllaç a l'arxiu anterior idèntic que ja té emmagatzemat. Git modela les seves dades més com a la Figura 1-5.

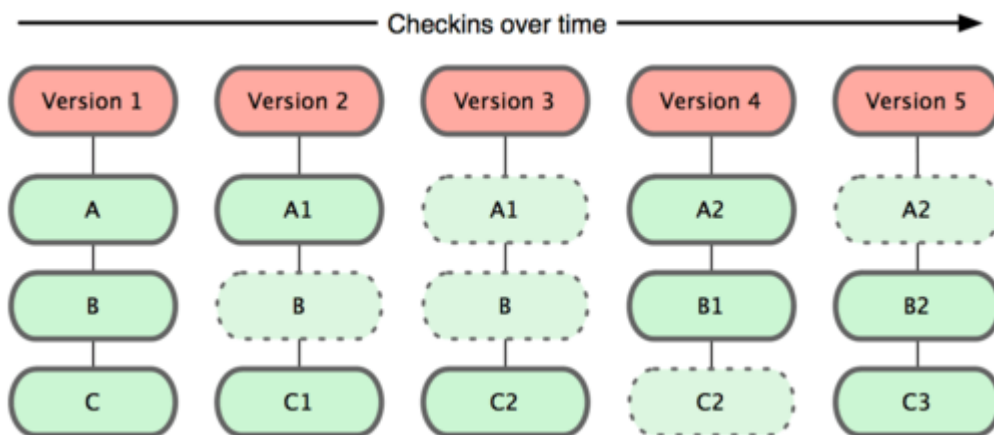


Figura 1-5. Git emmagatzema la informació com instantànies del projecte al llarg del temps.

Aquesta és una distinció important entre Git i pràcticament tots els altres VCSs. Fa que Git reconsideri gairebé tots els aspectes del control de versions que molts dels altres sistemes van copiar de la generació anterior. Això fa que Git s'assembli més a un mini sistema d'arxius amb algunes eines tremendament potents construïdes sobre ell, que a un VCS. Explorarem alguns dels beneficis que obtens al modelar les teves dades d'aquesta manera quan veiem ramificacions (branching).

Gairebé qualsevol operació és local

La majoria de les operacions en Git només necessiten arxius i recursos locals per operar. En general no es necessita informació de cap altre ordinador de la teva xarxa. Si estàs acostumat a un CVCS on la majoria de les operacions tenen aquesta sobrecàrrega del

retard de la xarxa, aquest aspecte de Git et farà pensar que els déus de la velocitat han beneït Git amb poders sobrenaturals. Com tens tota la història del projecte aquí mateix, en el teu disc local, la majoria de les operacions semblen pràcticament immediates.

Per exemple, per navegar per la història del projecte, Git no necessita sortir al servidor per obtenir la història i mostràrtela, simplement la llegeix directament de la base de dades local. Això vol dir que veus la història del projecte gairebé a l'instant. Si vols veure els canvis introduïts en un arxiu entre la versió actual i la de fa un mes, Git pot buscar l'arxiu fa un mes i fer un càlcul de diferències localment, en lloc d'haver de demanar-li a un servidor remot que ho faci, o obtenir una versió antiga des de la xarxa i fer-ho de manera local.

Això també significa que hi ha molt poc que no puguis fer si estàs desconnectat o sense VPN. Si et puges a un avió o un tren i vols treballar una mica, pots confirmar els canvis feliçment fins que aconseguixis una connexió de xarxa per pujar-los. Si te'n vas a casa i no aconseguixes que el teu client VPN funcioni correctament, pots seguir treballant. En molts altres sistemes, això és impossible o molt dolorós. En Perforce, per exemple, no pots fer molt quan no estàs connectat al servidor, i en Subversion i CVS, pots editar arxius, però no pots confirmar els canvis a la teva base de dades (perquè la teva base de dades no té connexió). Això pot no semblar gran cosa, però et sorprendria la diferència que pot suposar.

Git té integritat

Tot en Git és verificat mitjançant una suma de comprovació (checksum en anglès) abans de ser emmagatzemat, i és identificat a partir d'aquest moment mitjançant aquesta suma. Això significa que és impossible canviar els continguts de qualsevol arxiu o directori sense que Git ho sàpiga. Aquesta funcionalitat està integrada en Git al més baix nivell i és part integral de la seva filosofia. No pots perdre informació durant la seva transmissió o patir corrupció d'arxius sense que Git ho detecti.

El mecanisme que fa servir Git per generar aquesta suma de comprovació es coneix com hash SHA-1. Es tracta d'una cadena de 40 caràcters hexadecimals (0-9 i af), i es calcula en base als continguts de l'arxiu o estructura de directoris. Un hash SHA-1 té aquesta pinta:

24b9da6552252987aa493b52f8696cd6d3b00373

Veuràs aquests valors hash per tot arreu en Git, ja que els fa servir amb molta freqüència. De fet, Git no guarda tot per nom d'arxiu, sinó pel valor hash dels seus continguts.

Git generalment només afegeix informació

Quan realitzis accions en Git, gairebé totes elles només afegeixen informació a la base de dades de Git. És molt difícil aconseguir que el sistema faci alguna cosa que no es pugui desfer, o que d'alguna manera suprimeixi dades. Com en qualsevol VCS, pots perdre o malmetre canvis que no has confirmat encara, però després de confirmar una instantània en Git, és molt difícil de perdre, especialment si envies (push) la base de dades a un altre repositori amb regularitat.

Això fa que utilitzar Git sigui un plaer, perquè sabem que podem experimentar sense perill de fastiguejar greument les coses. Per a una anàlisi més exhaustiva de com emmagatzema Git seva informació i com pots recuperar dades aparentment perduts.

Els tres estats

Ara presta atenció. Això és el més important a recordar sobre Git si vols que la resta del teu procés d'aprenentatge prossegueixi sense problemes. Git té tres estats principals en els quals es poden trobar els teus arxius: confirmat (Committed), modificat (modified), i preparat (staged). Confirmat significa que les dades estan emmagatzemades de manera segura en la teva base de dades local. Modificat significa que has modificat l'arxiu però encara no ho has confirmat a la teva base de dades. Preparat significa que has marcat un arxiu modificat en la seva versió actual perquè vagi en la teva pròxima confirmació.

Això ens porta a les tres seccions principals d'un projecte de Git: directori de Git (Git directory), el directori de treball (working directory), i l'àrea de preparació (staging area).

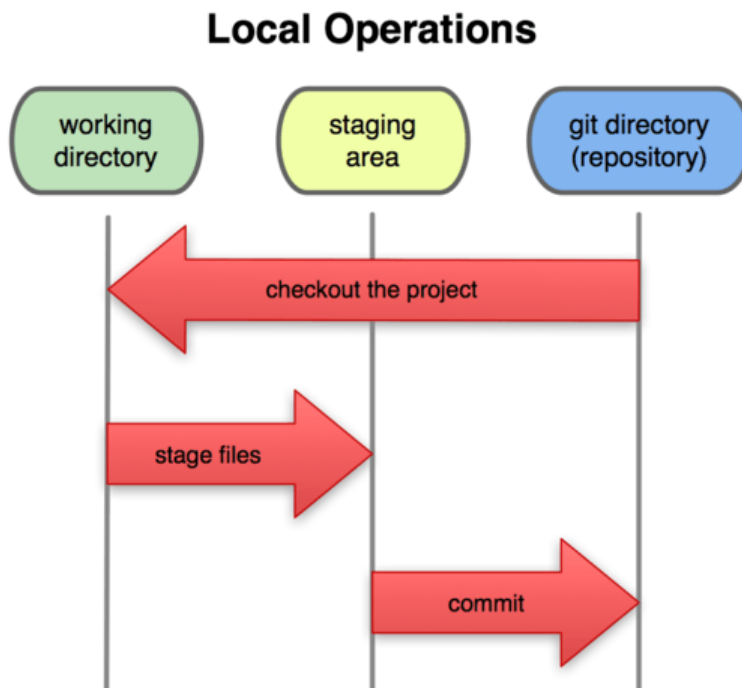


Figura 1-6. Directori de treball, àrea de preparació, i directori de Git.

El directori de Git és on Git emmagatzema les metadades i la base de dades d'objectes per al teu projecte. És la part més important de Git, i és el que es copia quan clones un repositori des d'un altre ordinador.

El directori de treball és una còpia d'una versió del projecte. Aquests arxius es treuen de la base de dades comprimida al directori de Git, i es col·loquen en disc perquè els puguis utilitzar o modificar.

L'àrea de preparació és un senzill arxiu, generalment contingut al directori de Git, que emmagatzema informació sobre el que anirà en la teva pròxima confirmació. De vegades se l'anomena índex, però s'està convertint en estàndard el referir-s'hi com l'àrea

de preparació.

El flux de treball bàsic en Git és una cosa així:

1. Modifiqueu una sèrie d'arxius en el teu directori de treball.
2. Prepara els arxius, afegint a la teva àrea de preparació.
3. Confirma els canvis, el que pren els fitxers tal com estan en l'àrea de preparació, i emmagatzema aquestes instantànies de manera permanent en el teu directori de Git.

Si una versió concreta d'un arxiu és al directori de Git, es considera confirmada (Committed). Si ha sofert canvis des que es va obtenir del repositori, però ha estat afegida a l'àrea de preparació, està preparada (staged). I si ha sofert canvis des que es va obtenir del repositori, però no s'ha preparat, està modificada (modified).

1.4 Instal·lant Git

Anem a començar a utilitzar una mica de Git. El primer és el primer: has de instal·lar. Pots obtenir-lo de diverses maneres, les dues principals són instal·lar-lo des de codi font, o instal·lar un paquet existent per a la teva plataforma.

Instal·lant des de codi font

Si pots, en general és útil instal·lar Git des de codi font, perquè obtindràs la versió més recent. Cada versió de Git tendeix a incloure útils millores en la interfície d'usuari, de manera que utilitzar l'última versió és sovint el camí més adequat si et sents còmode compilant programari des de codi font. També passa que moltes distribucions de Linux contenen paquets molt antics, així que a menys que estiguis en una distribució molt actualitzada o estiguis usant backports, instal·lar des de codi font pot ser la millor opció.

Per instal·lar Git, necessites tenir les següents llibreries de les que Git depèn: curl, zlib, openssl, expat, i libiconv. Per exemple, si estàs en un sistema que té yum (com Fedora) o apt-get (com un sistema basat en Debian), pots utilitzar aquestes comandes per instal·lar totes les dependències:

```
$ yum install curl-devel expat-devel gettext-devel \ openssl-devel  
zlib-devel $ apt-get install libcurl4-gnutls-dev libexpat1-dev gettext  
\ libz-dev
```

Quan tinguis totes les dependències necessàries, pots descarregar la versió més recent de Git des de la seva pàgina web:

<http://git-scm.com/download>

Després compila i instal·la:

```
$ tar -zxf git-1.6.0.5.tar.gz $ cd git-1.6.0.5 $ make  
prefix=/usr/local all $ sudo make prefix=/usr/local install
```

En aquest punt, també pots obtenir Git, a través del propi Git, per a futures actualitzacions:

```
$ git clone git://git.kernel.org/pub/scm/git/git.git
```

Instal·lant en Linux

Si vols instal·lar Git a Linux a través d'un instal·lador binari, en general pots fer-ho a través de l'eina bàsica de gestió de paquets que porta la distribució. Si estàs a Fedora, pots utilitzar yum:

```
$ yum install git-core
```

O si estàs en una distribució basada en Debian com Ubuntu, prova amb apt-get:

```
$ apt-get install git
```

Instal·lant en Mac

Hi ha dues maneres fàcils d'instal·lar Git en un Mac La més senzilla és usar l'instal·lador gràfic Git, que pots descarregar des de la pàgina de Google Code (vegeu Figura 1-7):

<http://code.google.com/p/git-osx-installer>

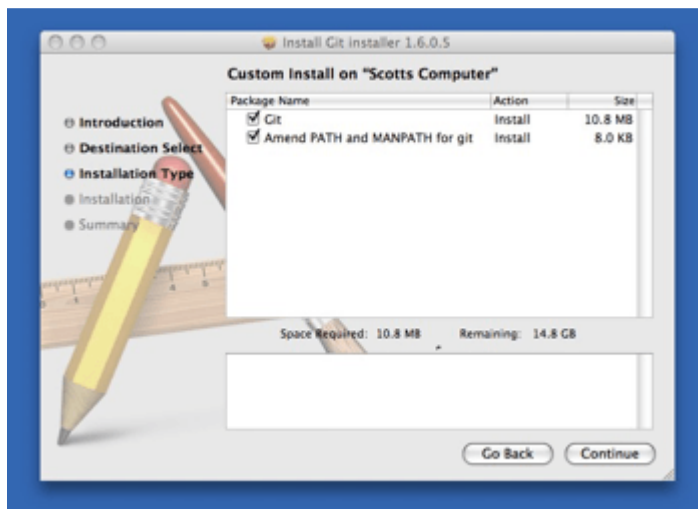


Figura 1-7. Instal·lador de Git per OS X.

L'altra manera és instal·lar Git mitjançant MacPorts (<http://www.macports.org>). Si tens MacPorts instal·lat, instal·la Git amb:

```
$ sudo port install git-core +svn +doc +bash_completion +gitweb
```

No necessites afegir tots els extrems, però probablement vulguis incloure + svn en cas que alguna vegada necessites utilitzar Git amb repositoris Subversion.

Instal·lant a Windows

Instal·lar Git en Windows és molt fàcil. El projecte msysGit té un dels processos d'instal·lació més senzills. Simplement descarrega el arxiu exe d'instal·lació des de la pàgina de GitHub, i executa'l:

<http://msysgit.github.com/>

Un cop instal·lat, tindràs tant la versió de línia d'ordres (inclòs un client SSH que ens serà útil més endavant) com la interfície gràfica d'usuari estàndard.

1.5 Configurant Git per primera vegada

Ara que tens Git en el teu sistema, voldràs fer algunes coses per personalitzar el teu entorn de Git. Només cal fer aquestes coses un cop, es mantindran entre actualitzacions. També pots canviar en qualsevol moment tornant a executar les ordres corresponents.

Git porta una eina anomenada `git config` que et permet obtenir i establir variables de configuració, que controlen l'aspecte i funcionament de Git. Aquestes variables poden emmagatzemar en tres llocs diferents:

- Arxiu `/etc/gitconfig` : Conté valors per a tots els usuaris del sistema i tots els seus repositoris. Si passes l'opció `--system` a `git config`, llegeix i escriu específicament en aquest arxiu.
- Arxiu `~/.gitconfig` file: Específic al teu usuari. Pots fer que Git llegeixi i escrigui específicament en aquest arxiu passant l'opció `--global`.
- Arxiu `config` al directori de git (és a dir, `.git/config`) del repositori que utilitzeu actualment: Específic a aquest repositori. Cada nivell sobreescriu els valors del nivell anterior, de manera que els valors de `.git/config` tenen preferència sobre els de `/etc/gitconfig`.

En sistemes Windows, Git busca l'arxiu `.gitconfig` al directori `$HOME` (`C:\Documents and Settings\USER` per a la majoria d'usuaris). També busca en el directori `/etc/gitconfig`, encara que aquesta ruta és relativa a l'arrel `MSys`, que és allà on decidís instal·lar Git en el teu sistema Windows quan vas executar l'instal·lador.

La teva identitat

El primer que hauries de fer quan instal·les Git és establir el teu nom d'usuari i adreça de correu electrònic. Això és important perquè les confirmacions de canvis (commits) en Git usen aquesta informació, i és introduïda de manera immutable en els commits que envies:

```
$ git config --global user.name "John Doe" $ git config --global user.email johndoe@example.com
```

De nou, només et cal fer això una vegada si especifiques l'opció `--global`, ja que Git sempre utilitzarà aquesta informació per a tot el que facis en aquest sistema. Si vols sobreescriure aquesta informació amb un altre nom o adreça de correu per a projectes específics, pots executar la comanda sense l'opció `--global` quan estiguis en aquest projecte.

El teu editor

Ara que la teva identitat està configurada, pots triar l'editor de text per defecte que s'utilitzarà quan Git necessiti que introdueixis un missatge. Si no indiques res, Git utilitza l'editor per defecte del sistema, que generalment és `Vaig veure` o `Vim`. Si vols utilitzar un altre editor de text, com `Emacs`, pots fer el següent:

```
$ git config --global core.editor emacs
```

La teva eina de diferències

Una altra opció útil que potser vulguis configurar és l'eina de diferències per defecte, usada per resoldre conflictes d'unió (merge). Diguem que vols utilitzar vimdiff:

```
$ git config --global merge.tool vimdiff
```

Git accepta kdiff3, tkdiff, MELD, xxdiff, emergeix, vimdiff, gvimdiff, ecmerge, i opendiff com a eines vàlides. També pots configurar l'eina que tu vulguis, vegeu el capítol 7 per a més informació sobre com fer-ho.

Comprovant la configuració

Si vols comprovar la teva configuració, pots utilitzar la comanda `git config --list` per llistar totes les propietats que Git ha configurat:

```
$ git config --list user.name=Scott Chacon
user.email=schacon@gmail.com color.status=auto color.branch=auto
color.interactive=auto color.diff=auto ...
```

Potser vegis claus repetides, perquè Git llegeix la mateixa clau de diferents arxius (`/etc/gitconfig` i `~/.gitconfig`, per exemple). En aquest cas, Git utilitza l'últim valor per a cada clau única que veu.

També pots comprovar quin valor creu Git que té una clau específica executant `git config {clave}`:

```
$ git config user.name Scott Chacon
```

Extret de: <http://git-scm.com/>